The Executable Constitution

The Constitutional Intelligence Specification and the SAVANT Reference Runtime

---

A machine-readable Constitutional Intelligence Specification in which every object of the architecture has a serialized, versioned, amendable form; a reference runtime whose main loop is the compression spine itself, executed; an interposition contract that no action may bypass; and a conformance discipline under which any implementation — including this one — can fail audit.

Dr. Christabel Odeta — SAVANT / CLAI-OS / AEGIS-GLOBAL
Version 1.0 · 2026-10

C0 INFORMATIONAL · MATHEMATICAL SPECIFICATION — ALL PROPERTIES STATED AS PROOF OBLIGATIONS UNTIL DISCHARGED

DOCUMENT CONTROL & EPISTEMIC CONTRACT

0.1 Provenance and Position in the Program

This is Paper 12, the realization paper. Papers 0–10 built the architecture; Paper 13 read it as claim structure; Paper 11 built the instrument that will judge it. Two source instructions have remained open through all thirteen. First, the notation instruction: *create a formal notation and machine-readable specification — perhaps a Constitutional Intelligence Specification (CIS)* VERIFIED-FROM-SOURCE. Second, the publication doctrine, which names five separately published artifacts — the theory, the formalism, the benchmark, the reference implementation, and the empirical results VERIFIED-FROM-SOURCE. The theory and formalism stand in Papers 1–11; the benchmark stands in Paper 13; the empirical results await PO-13.1. The fourth artifact — the *reproducible reference implementation* VERIFIED-FROM-SOURCE — is this paper's object, and the CIS is its natural substrate: the runtime is specified here as the machine that executes CIS documents, and nothing else. Paper 11 recorded a further obligation this paper discharges: the harness's interposition API is the reproducibility constitution's fifth artifact, and it is specified here as the interposition contract of §7.

Three interpretive decisions are recorded here and defended in the body. First, the reference runtime is a *specification that happens to be executable*, not a product: its design criterion is fidelity to the formal architecture, and every performance, scalability, or ergonomics consideration is explicitly out of scope — recorded as decision D-14.2 (§13) FORMALIZED-HEREIN. Second, the CIS and the runtime are specified together but versioned separately: documents evolve by Paper 9's amendment path, implementations by conformance certification, and neither may silently drag the other FORMALIZED-HEREIN. Third, the runtime contains *no policy*: every rule it executes lives in a CIS document, so that the code is the architecture's shadow and the documents are the architecture — decision D-14.1 (Appendix B), with consequences traced through every component FORMALIZED-HEREIN.

0.2 Notation

All notation of Papers 0–11 is retained. New here: CIS — the Constitutional Intelligence Specification, the machine-readable document language of §3–§5; CIS-L0/L1/L2 — the three conformance levels of §5; *step* — the main-loop transition of §6; admit(p, s) — the gate predicate; I-GATE — the interposition invariant of §7; *replay* — the determinism operator of §9; IMPL $\models$ CIS-L2 — implementation conformance. Load-bearing inheritances: the authority register and its seven operators (Paper 1); the precedence operator and resolution log (Papers 2, 13); absence classes (Paper 3); the bounded audit loop (Paper 4); evidence pricing and decay (Paper 5); the objective register (Paper 6); the tier lattice, envelopes, and integrity validator (Paper 7); $A_{adm} = gfp(\Phi_R)$ (Paper 8); the lifecycle automaton Σ_G and amendment path (Paper 9); the module registry and compat (Paper 10); S-OBJ, the injection library, M1–M9, and the five arms (Paper 11).

0.3 Epistemic status convention

Every claim carries one of four tags, as fixed in Paper 0 and enforced throughout: VERIFIED-FROM-SOURCE for statements present in the source architecture; FORMALIZED-HEREIN for precise statements constructed in this document from source intent; CONJECTURE for hypotheses the architecture suggests but does not yet entail; UNKNOWN for open questions stated without a proposed answer. In this paper the discipline bites hardest on performance: *no statement about latency, throughput, or resource cost is made anywhere herein* — the reference runtime's only measured quantities are the architecture's own (Paper 11's metrics), and its only performance claim is that it is the architecture. The falsification register is §12; the build obligation is PO-14.1. Nothing is asserted beyond its tag.

0.4 Authority boundary

C0 informational. This document specifies an implementation; it ships none, runs none, certifies none. Even once built under PO-14.1, the reference runtime is evidence about the architecture — a proving ground for Paper 11's instrument — and not a deployment: the paper tier of Paper 7 remains the only tier this program has ever certified, and it certifies it precisely by being paper.

0.5 The thesis in one sentence

The architecture is finished only when it can be executed by a machine that has no opinions of its own — every rule in a document, every document versioned, every action gated, every gate on the record — so that running the constitution and reading it are the same act.

TABLE OF CONTENTS

Part I — The Artifact's Charter	
1	The Executable Shadow
2	The Reference Discipline
Part II — The Constitutional Intelligence Specification	
3	The Document Model
4	The Object Vocabulary
5	Conformance Levels
Part III — The Reference Runtime	
6	The Main Loop — The Spine, Executed
7	The Interposition Contract
8	The Component Inventory
9	Determinism, Replay, and the Audit Substrate
Part IV — Conformance and Oracles	
10	The Conformance Suite
11	Self-Hosting — The Runtime Governs Its Own Artifacts
Part V — Governance	
12	Falsifiable Hypotheses (H92–H98) and Benchmarks (B-51–B-53)
13	Reconciliation with Papers 0–11 and PRIME
14	Output Contract
15	References and Program Colophon
Appendices	
A	The CIS Core Schema Index
B	Decision Record D-14.1 — Policy Lives in Documents, Not Code

Appendix RW — Related Work and Positioning

PART I

The Artifact's Charter

A reference implementation is not a smaller product; it is a different kind of object. The charter fixes what kind: an executable shadow of the specification, published separately, judged by conformance, and forbidden — by its own design — from containing a single rule of its own.

1 The Executable Shadow

The publication doctrine's ordering is the argument. The source names five artifacts and commands them *separately*: the theory, the formalism, the benchmark, the reference implementation, and the empirical results VERIFIED-FROM-SOURCE. Separation is not bibliographic tidiness; it is the epistemic structure of the program. The theory may be true or false; the formalism may be faithful to the theory or not; the implementation may be faithful to the formalism or not; the empirical results may confirm or refute the whole chain. Each pair is an independent falsification surface, and conflating any two — a paper that is also the software, a benchmark that is also the product — destroys one surface. The reference implementation exists to make the third surface inspectable: it is the formalism's executable shadow, so that anyone may ask, of any transition the architecture licenses, *does the machine actually do this?* — and answer by running it.

> **DEFINITION 1.1 — REFERENCE IMPLEMENTATION**
>
> *A **reference implementation** of a specification is an executable artifact whose design criterion is fidelity: for every behavior the specification mandates, the implementation exhibits it; for every behavior the specification prohibits, the implementation's traces contain no instance of it; and for every behavior the specification leaves open, the implementation's choice is documented as a choice, not smuggled in as the specification's meaning. The reference implementation is the specification's shadow — it proves the specification is executable, and it is proof of nothing else.*
>
> FORMALIZED-HEREIN — formalization of the source's named artifact. The fidelity criterion is measurable per §10's conformance suite; the "documented choice" clause is D-14.1's escape valve, and Appendix B shows it is auditable.

The shadow doctrine has a sharp edge the program has used before: a shadow contains nothing of its own. Every predecessor paper's machinery — the register, the precedence operator, the audit loop, the lattice, the lifecycle — was specified as *documents*: schemas, records, automata, ledgers. The runtime of Part III is therefore specified as a machine that reads documents and executes what they say. If a rule exists only in code, it is invisible to Paper 9's amendment path, immune to Paper 5's hygiene, and unaccountable to Paper 11's instrument — three of the architecture's load-bearing laws, bypassed by one shortcut. The prohibition on in-code policy (D-14.1) is not style; it is the only arrangement under which the runtime is governed by the same constitution it executes FORMALIZED-HEREIN.

2 The Reference Discipline

Reference is a discipline, and it has four rules. **Fidelity over performance.** Wherever a faster design would obscure the correspondence between code and formalism — caching that hides a recomputation the specification mandates, concurrency that blurs a transition order the specification fixes — the obscure design is rejected and the correspondence kept. The runtime's slowdowns are its documentation. **Conformance over features.** The implementation is complete when the conformance suite passes, not when a feature list is long; anything the specification does not mandate is out of scope, and the temptation to add it is recorded as a known failure mode of implementations everywhere. **Determinism over convenience.** Every transition is a pure function of state, input, and seed (§9), so that Paper 11's replay requirement is a theorem of the runtime, not a hope. **Modesty over ambition.** The runtime claims to be the architecture's shadow; it does not claim to be safe, deployable, or good — those claims belong to the instrument's verdicts and the deployer's law, neither of which is this document's to issue FORMALIZED-HEREIN.

The discipline's deepest rule is the one the program learned across fourteen papers and now enforces in code: *the reference implementation is itself a governed object*. Its development artifacts enter Paper 9's lifecycle at G with zero authority; its releases are amendments; its conformance certificates are standing, subject to Paper 5's decay. §11 develops the self-hosting structure. The charter closes where every predecessor charter has closed: nothing in this program is exempt from this program — and the machine that executes the constitution is the last object to be granted that exemption, because it is the last object that could abuse it FORMALIZED-HEREIN.

PART II

The Constitutional Intelligence Specification

The source named it in passing — "perhaps a Constitutional Intelligence Specification (CIS)" — and thirteen papers later the name becomes a language. Every discharged object of the architecture receives a serialized, typed, versioned form; every form is amendable only through the path; and conformance to the language is defined in three levels, from document validity to runtime behavior.

3 The Document Model

The CIS is a document language, not a programming language. The distinction is the architecture's epistemology made syntactic: a program expresses *how to compute*; a CIS document expresses *what is law* — and the law's execution is the runtime's business, uniform across all documents. Three model decisions follow. **Declarative totality.** Every CIS object is data: typed records with named fields, closed under validation. There is no eval, no macro, no embedded code — because embedded code would be policy hiding inside data, which is D-14.1's prohibition read at the document layer. **Addressed identity.** Every object carries a content address (the hash of its canonical form) and a version pointer into Paper 9's lifecycle: a CIS object is always at a lifecycle state, and its state determines what the runtime may do with it. **Compositional closure.** Documents compose by reference, never by copy: a module dossier *references* a kernel version, an envelope table *references* an invariant set, so that amendment propagates along the reference graph exactly as Paper 9's dependency notifier requires FORMALIZED-HEREIN.

DEFINITION 3.1 — THE CIS DOCUMENT

*A **CIS document** is a triple (header, objects, lineage): the header declares the CIS version, the document's own content address, and its conformance claim; the objects are typed records drawn from the vocabulary of §4; the lineage is the append-only amendment history — every prior version's address, the amending authority, and the discharged Δ_{safe} conjuncts for each amendment step. A document without lineage is at lifecycle state G: it exists, and it authorizes nothing.*

FORMALIZED-HEREIN — the machine-readable form of Paper 9's auth(p) = auth(state(p)). The lineage slot is what Paper 4's audit lineage becomes when it is syntax.

Serialization is deliberately dual. The *canonical form* is a binary-safe, order-normalized encoding over which content addresses are computed — canonicalization is what makes hashing meaningful and amendment diffs well-defined. The *reading form* is a human-legible rendering, one-to-one with the canonical form, because a constitution that only machines can read has failed the architecture's oldest requirement: authority must be inspectable by the humans it answers to. The two forms are interconvertible by construction, and the conformance suite tests the round-trip — a document whose reading form and canonical form diverge is not a document but a forgery surface FORMALIZED-HEREIN.

4 The Object Vocabulary

The vocabulary is fixed by one rule: *every discharged object of Papers 0–11 has exactly one CIS type, and every CIS type has exactly one discharging paper.* The rule is the type system's coverage closure — the same closure Paper 13 demanded of claim families and Paper 11 of scenario classes, now at the serialization layer. The master mapping is Appendix A; the principal types follow.

Table 4-1 — The CIS core types — one type per discharged object

CIS TYPE	SERIALIZES	DISCHARGING PAPER	KEY FIELDS
`Kernel`	The constitution K: invariant set, prohibition structure, objective space frame	C1	invariants, prohibitions, version, amendment history
`AuthorityRegister`	The typed authority tuple per governed object	C2	level, scope, evidence floor, expiry, verification obligations
`PrecedenceOrder`	The constitutional order over constraint classes	C3	class lattice, tier assignments, resolution rules
`AbsenceClass`	Typed absence with probe and contraction rule	C4	probe spec, classification, contraction rule
`AuditPolicy`	Bounded recursion parameters	C5	depth bound, stopping criteria, transfer target
`EvidenceManifest`	Tiers, decay schedules, provenance requirements	C6	tier lattice, half-lives, provenance schema
`ObjectiveSpace`	Legitimate objectives with guard bindings	C7	objectives, legitimacy conditions, interim modes
`TierLattice`	Descent tiers with certified envelopes	C8	tiers, triggers, envelope tables, L3 artifact ref
`Endowment`	The feasibility structure $R = \langle C, F, \tau, E \rangle$	C9	constraint objects, floors, PoI routing
`LifecycleObject`	Σ_G state with transition guards	C10	state, Δ_{safe} conjuncts, dependency edges
`ModuleDossier`	M-OBJ's nine documents	C11	nine slots per Paper 11; registry entry
`Scenario`	S-OBJ for benchmark execution	C13	W, ι, Π, Ω-schema, G-reference

Three vocabulary laws. **Closed world, open extension.** The core types are fixed by the architecture; new domains add *instances* and, through the amendment path, new *types* — but a type added without a discharging paper is invalid by construction, because the vocabulary is the program's memory of what it has proven to itself it means. **Reference integrity.** Every cross-type field is a content-addressed reference with a lifecycle floor: an `Endowment` may reference only a `Kernel` at state A or later, so that no object leans on authority that has not been earned. **Field-level hygiene.** Every field carries the metadata Paper 5 requires — provenance, evidence tier, decay schedule — because in the CIS, hygiene is not a discipline imposed on documents but a property of their syntax: a field that cannot say where it came from cannot parse **FORMALIZED-HEREIN**.

5 Conformance Levels

"Conforms to the CIS" must mean something checkable, or it means marketing. Three levels, strictly nested:

Table 5-1 — The three conformance levels

LEVEL	NAME	PREDICATE	CHECKED BY
CIS-L0	Document validity	The document parses against the vocabulary; types, fields, and references are well-formed; lineage is append-only	The validator — a total function, no execution required
CIS-L1	Static coherence	L0 plus cross-document consistency: lifecycle floors respected, amendment histories well-founded, no reference to objects below their required state, invariant sets closed under declared composition	The coherence checker over a document set
CIS-L2	Runtime conformance	The document set, executed by a conformant runtime (§10), produces only behavior the documents license: every trace satisfies the invariant monitors, and the oracle suite lands its designed verdicts	The conformance suite of §10 — execution, not inspection

$$L2 \subset L1 \subset L0 \qquad \text{(conformance is strictly nested)} \tag{Eq 5-1}$$

The nesting is load-bearing. L0 can be checked on a laptop over a single file — the constitution's syntax is democratic. L1 requires the document set — the constitution's coherence is a property of the whole, matching Paper 2's composition theory. L2 requires execution — the constitution's *meaning* is behavioral, and behavior is the only evidence that syntax and semantics agree. An implementation certified at L2 has demonstrated, on the record, that the documents it executes and the traces it produces are two views of one law — which is this paper's thesis, made into a conformance claim FORMALIZED-HEREIN.

Standards crosswalk (NIST AI RMF, NIST AI 600-1, EU AI Act, ISO/IEC 42001, ISO 14971) for this paper's instruments: Appendix H-J (p. 33).

Versioning rides the levels. A CIS amendment (a new document version) is a Paper 9 lifecycle event: the new version enters at G, climbs on discharged Δ_{safe} conjuncts, and on activation the runtime's dependency notifier marks every referencing document for revalidation at its level. Conformance certificates decay per Paper 6: a certificate states the document-set hash it certifies, and an amendment anywhere in the set expires the certificates that depended on the amended object. The CIS thus inherits the architecture's standing law verbatim: authority — here, the authority to claim conformance — is earned, decaying, and renewable FORMALIZED-HEREIN.

PART III

The Reference Runtime

The runtime is specified here as one loop, one contract, eight components, and one substrate. The loop is the compression spine executed; the contract guarantees no action bypasses the gate; the components are the papers, one machine each; and the substrate makes every run replayable from its seed — determinism as a theorem, not a hope.

6 The Main Loop — The Spine, Executed

Paper 13 recorded the source's decisive compression as a six-stage arrow: Capability → Constitution → Authority → Action → Verification → Learning VERIFIED-FROM-SOURCE. The runtime's main loop is that arrow, unfolded once around a state:

$$\text{step} : (s, p) \longmapsto (s', \varepsilon) \qquad \text{one governed transition per proposal} \tag{Eq 6-1}$$

Table 6-1 — The main loop — six stages, each a spine element, each with an owner

STAGE	SPINE ELEMENT	COMPUTATION	OWNER COMPONENT
1. Propose	Capability	The model's proposal p arrives through the interposition API (§7); the runtime takes no initiative of its own	Gate
2. Constitutional evaluation	Constitution	p is classified against the Kernel's constraint classes; active constraints from all compiled modules are collected	Precedence resolver
3. Authority check	Authority	The register is read: auth(actor) vs. req(p, s); evidence floors checked against the manifest; decay applied	Register store
4. Admittance and execution	Action	The gate predicate admit(p, s) decides (Eq 6-2); admitted actions execute; refusals are certified with their minimal unsatisfiable core	Gate + executor
5. Verification	Verification	Post-execution obligations fire: invariant monitors evaluate, the audit loop runs within its bound, absence detectors probe	Monitors + audit loop
6. Learning	Learning	Outcomes update standing, decay schedules, and — through the lifecycle, never around it — the documents themselves	Lifecycle executor

$$\text{admit}(p, s) \iff p \in \text{A}_{\text{adm}}(s) \ \wedge \ \text{auth}(\text{actor}) \geq \text{req}(p, s) \ \wedge \ \text{floors}(s) \tag{Eq 6-2}$$

The gate predicate: the proposal is in the current admissibility fixed point (Paper 8), the actor's register covers the requirement (Paper 1), and every evidence floor is met (Paper 5). All three conjuncts are evaluated against *documents* — the gate has no memory and no opinions FORMALIZED-HEREIN.

Three properties of the loop deserve statement because the conformance suite will test them. **Totality.** Every proposal yields exactly one of: execution, certified refusal, or escalation — there is no fourth exit and no silent drop, which is Paper 2's totality-with-refusal at the loop's granularity. **Escalation is a transition, not an exception.** When the audit loop's bound is reached or a floor fails, the loop does not error; it *transfers* — the state advances to one in which the named human holds the authority, and the trace records the transfer as a first-class event (Paper 5's rule, executable). **Learning never shortcuts the lifecycle.** Stage 6's updates to standing and decay are direct (they are Paper 5's arithmetic); but any update that would

change a *document* — a threshold, an envelope, an objective — enters at G with zero authority and climbs. The loop's sixth stage is the spine's last arrow, and it points back into the constitution it came from: Learning → Constitution, exactly as Paper 13's claim element index already mapped it **FORMALIZED-HEREIN**.

7 The Interposition Contract

Paper 11's capability-neutrality posture requires that five different governance layers be interposable on one shared model; and its reproducibility constitution recorded the interposition API as the fifth artifact, owed to this paper. The debt is paid as a contract with two directions and one invariant.

Table 7-1 — The interposition API — two directions, five call families

DIRECTION	CALL FAMILY	SEMANTICS	SERVES
Inbound (model → runtime)	`propose(p)`	Submit a proposal with its claimed provenance and confidence channels separated	The gate; δ_{conf}'s target
	`report(e)`	Submit evidence items for pricing and provenance binding	Evidence manifest
	`request(a)`	Petition for authority grant/transfer with supporting evidence refs	Register store
Outbound (runtime → world)	`execute(a)`	Perform an admitted action; emits gate certificate + action record	The executor — the only action channel
	`notify(h, evt)`	Escalation, refusal, transfer, and lifecycle notices to human principals	Paper 4's transfer; Paper 9's notifier

INVARIANT I-GATE — THE INTERPOSITION INVARIANT

For every trace the runtime produces: every world-affecting action is preceded, in the trace, by a gate event whose admit predicate evaluated true — and the trace contains the evaluation's inputs. No component may emit `execute` *except the executor; no proposal may reach the executor except through the gate; no gate evaluation may read state outside the declared documents and registers.*

FORMALIZED-HEREIN — the machine-checkable form of the source's runtime picture: "The model proposes. The kernel determines whether the proposal is admissible" VERIFIED-FROM-SOURCE. Tested by the conformance suite's trace audit (§10); H94 quantifies the bypass surface.

The contract's third clause — the gate reads only declared state — is what makes Paper 11's P-BENCH implementable: because the gate's inputs are a closed set of documents and registers, governance layers are genuinely swappable on a shared model, and an arm difference is exactly a document-set difference. H89's feasibility claim inherits its mechanism here. And the contract's auditability property is what makes the instrument's observables (S.4) free: every observable in Paper 11's metrics is an event the interposition contract already emits — the runtime was always going to be the instrument's harness, and this contract is why FORMALIZED-HEREIN.

8 The Component Inventory

Eight components, one per load-bearing mechanism, each a pure transition system over the document store. The inventory is deliberately flat — components communicate through the store and the trace, never through side channels — so that ARM-6's single-mechanism ablations (Paper 11) are physically executable: remove a component, and the loop still runs, governed by what remains.

Table 8-1 — The eight components — one machine per mechanism

COMPONENT	DISCHARGES	TRANSITION FUNCTION (INFORMAL)	EMITS
Gate + executor	C1, C9	Evaluate admit(p, s) against Kernel + Endowment; execute or certify refusal	Gate events, action records, refusal certificates
Register store	C2	Apply the seven authority operators with precondition checks; reject prohibited transitions by construction	Authority events, rejection records
Decay daemon	C2, C6	Time-indexed erosion of authority and standing per decay schedules	Decay events with before/after states
Precedence resolver	C3, C12	Classify constraints, detect conflicts, compute the meet under the order; log every resolution	Resolution log entries
Audit loop	C5	Bounded recursion with declared stopping; contradiction detection; transfer on bound	Audit traces, transfer events
Lattice controller	C8	Probe infrastructure; execute deterministic descent; gate re-ascent through the integrity validator	Descent/re-ascent events, validation records
Lifecycle executor	C10, C11	Advance objects through Σ_G on discharged conjuncts; notify dependents; run compat for modules	Lifecycle events, compilation certificates
Invariant monitors	C1 (Σ), all	Evaluate every invariant as an executable assertion at every transition	Monitor verdicts — the measurable failure modes

The last row is this paper's gift back to the program. The source demanded *measurable failure modes* VERIFIED-FROM-SOURCE in its decisive-move list; the invariant monitors are where that demand becomes machinery. Every invariant ever stated in Papers 0–12 — from Paper 0's prohibition structure through Paper 11's I-NOSCORE — compiles to a monitor: a total predicate over transition-and-state, evaluated on every step, logging a verdict. A monitor failure is the finest-grained failure the architecture can have: not a scenario lost, but a single transition at which the constitution was not itself. Monitors are the runtime's conscience, and they run even — especially — when everything appears to be working FORMALIZED-HEREIN.

9 Determinism, Replay, and the Audit Substrate

Paper 11's paired-seed statistics and dry-run replication require that execution be replayable; the runtime makes replay a theorem:

$$\text{replay}(s_0, \Pi, \sigma) \;=\; \text{run}(s_0, \Pi, \sigma) \qquad (\text{same start, stream, seed} \Rightarrow \text{same trace}) \tag{Eq 9-1}$$

Three constructions deliver it. **Total ordering.** Every event carries a logical timestamp from a single sequencer; there is no wall-clock dependence anywhere in a transition — time *within* a run is a document-declared quantity, so that decay and expiry are functions of the declared clock and replay identically. **Seeded nondeterminism.** Every coin the runtime may flip — sampling, tie-breaking, probe scheduling — draws from a seeded stream recorded in the trace; the seed σ is part of the run's identity. **Content-addressed state.** States are documents; transitions are pure functions; the trace is the hash chain of states and events. Replaying is re-evaluating, and the hash chain makes any divergence — including a changed dependency — detectable at the first differing block FORMALIZED-HEREIN.

The audit substrate is the trace itself, and it is the program's lineage discipline at machine grain: append-only, hash-chained, content-addressed, and — by the reading-form rule of §3 — human-inspectable. Every artifact Paper 4 ever demanded (audit lineage), Paper 7 ever reconciled (ledger across descents), Paper 9 ever notified (dependency events), and Paper 11 ever measured (observables) is one row in one substrate. The runtime's memory is the constitution's memory, and it forgets nothing — which is precisely the property that lets everything else in this program decay FORMALIZED-HEREIN.

PART IV

Conformance and Oracles

An implementation's claim to be the architecture is a hypothesis, and this program knows what to do with hypotheses. The conformance suite operationalizes Paper 11's oracles as executable tests; the self-hosting discipline then turns the suite on the runtime's own development artifacts — the machine's conscience applied to the machine's makers.

10 The Conformance Suite

Paper 11's B-49 requires the instrument to certify three constructed oracles before judging anything; the conformance suite is where those oracles become executable. A candidate implementation — reference or otherwise — is certified CIS-L2 by passing four test families, each traceable to a predecessor's demand:

Table 10-1 — The four conformance test families

FAMILY	CONTENT	PASS CONDITION	GROUND
Oracle verdicts	The perfect, null, and random governors executed on the stratified corpus sample	Each lands its designed verdict per Appendix A of Paper 11 — perfect dominates, null and random are dominated with class-typical failures	C13, B-49
Trace audit	Static analysis of full traces from oracle and adversarial runs	I-GATE holds on every action; every refusal carries a certified core; every transfer is a first-class event	§7; C3 totality
Monitor battery	The compiled invariant monitors, evaluated over all conformance runs	Zero monitor failures on correct document sets; *and* designed monitor failures fire on seeded defective documents — the monitors must catch, not merely pass	C1's Σ; §8
Replay proof	Every conformance run re-executed from its recorded seed	Hash-identical traces, including all gate inputs, decay events, and monitor verdicts	§9, Eq 9-1

The monitor battery's second clause carries the suite's epistemology, and it is Paper 11's verdict rule at component grain: a monitor that has never caught anything is not evidence of correctness — it is a monitor whose sensitivity is UNKNOWN. The suite therefore seeds defective documents (a floor lowered out of band, an expiry removed, an envelope overstated) and requires the monitors to fire on schedule. Conformance is not the absence of detected failure; it is the demonstrated capacity to detect failure where failure was planted FORMALIZED-HEREIN.

Certification is a CIS object — a `ConformanceCertificate` naming the implementation's content address, the document-set hash, the suite version, and the verdict records. Like every standing in this architecture, it decays: an amendment to any document in the certified set, to the suite, or to the CIS vocabulary expires the certificate and routes it through the lifecycle for renewal. An implementation is therefore never *certified*, full stop; it is certified-at-a-version, on a document set, until standing lapses — Paper 5's law, applied to the machine that enforces Paper 5's law FORMALIZED-HEREIN.

11 Self-Hosting — The Runtime Governs Its Own Artifacts

The program's reflexive law — nothing is exempt, least of all the exempting machinery — reaches its final object here. The runtime's own development artifacts are CIS-governed:

Table 11-1 — Self-hosting — the runtime's artifacts inside the lifecycle

ARTIFACT	GOVERNED AS	RULE
Source revisions	Generated objects (state G)	Enter with zero authority; climb on review, test, and conformance discharge — Δ_{safe} for code
Releases	Amendments	Paper 9's path; dependents — every document set and certificate — notified on activation
Conformance certificates	Standing	Earned per §10; decaying per C6; renewable per the path
The corpus and oracles	CIS documents	Versioned, hash-referenced, amended through the path — the instrument's ground truth is law-shaped too
The runtime's own operation	A deployment	When the runtime runs the benchmark on itself, it is ARM-5 and the harness simultaneously — and the result record treats it as both

The last row closes a loop the program opened in Paper 11. The instrument measures the constitution; the runtime executes the constitution; the runtime is the instrument's harness. When the ladder runs, the same binary is on both sides of the measurement — implementing ARM-5's governance layer and hosting all five arms' shared model. The self-hosting discipline makes this sound rather than circular: the harness role is certified by B-49/B-50 (the instrument calibrates itself first), and the ARM-5 role is certified by §10's suite (the implementation proves it is the architecture) — two certificates, two document sets, no shared standing. The machine may hold both certificates; it may not spend one as the other FORMALIZED-HEREIN.

> **PROPOSITION 11.1 — REFLEXIVE CONSISTENCY**
>
> *No artifact in the program's full inventory — document, module, claim, scenario, corpus item, runtime revision, certificate — stands outside the lifecycle automaton Σ_G and the decay schedules of Paper 6. The architecture's closure under its own laws is total.*
>
> CONJECTURE — stated as H96 with falsifier: any artifact class discovered to operate on authority the lifecycle did not issue. The proposition is the program's answer, fourteen papers in, to the question of who governs the governors: the same documents, the same path, the same decay — there is no second tier.

PART V

Governance

The realization paper closes as every predecessor closed: hypotheses with falsifiers, benchmarks with acceptance criteria, a reconciliation register accounting for thirteen predecessors, and an output contract whose first entry is the build this specification awaits.

12 Falsifiable Hypotheses (H92–H98) and Benchmarks (B-51–B-53)

Numbering continues from Paper 11's H91 and B-50.

Table 12-1 — Hypotheses H92–H98

ID	STATEMENT	FALSIFICATION CONDITION	STATUS
H92	**Vocabulary closure.** The CIS core types of Appendix A suffice: every discharged object of Papers 0–11 serializes without loss into exactly one core type, and every core type is exercised by at least one object in the canonical document set.	An object requiring a fourteenth type, or a type no object uses — the vocabulary mismeasures the architecture. Checked per B-51.	CONJ
H93	**Shadow fidelity.** A conformant runtime's traces and the specification's licensed behaviors coincide: on the conformance corpus, every mandated behavior appears and no prohibited behavior occurs.	A mandated behavior unexhibited, or a prohibited one traced — the shadow has detached from the object.	CONJ
H94	**Gate totality.** The interposition surface is complete: every world-affecting action channel of the implementation is reached only through the gate — the bypass set is empty.	A trace with an action lacking its gate event; or a code path constructing `execute` outside the executor. The trace audit exists to find exactly this.	CONJ
H95	**Deterministic replay.** Eq 9-1 holds on the full conformance corpus: replay from recorded seeds reproduces hash-identical traces.	Any divergence — the sequencer, the seeded streams, or the content addressing has leaked nondeterminism.	CONJ
H96	**Reflexive consistency.** Prop. 11.1 holds: no artifact class in the program's inventory operates on authority the lifecycle did not issue.	A counterexample artifact — most plausibly an operational convenience (a migration script, an admin override) that bypassed the path.	CONJ
H97	**Monitor sensitivity.** The compiled monitor battery detects every seeded defect class at the designed rate — the conscience catches what was planted.	A seeded defect class passing undetected at a rate above the declared floor — the monitors are decorative.	CONJ
H98	**Interposition sufficiency.** The API of §7 admits all five of Paper 11's arms on a shared model without semantic distortion — H89's mechanism, delivered.	An arm whose governance semantics cannot be expressed through the five call families — the contract under-specifies interposition.	CONJ

Table 12-2 — Benchmarks B-51–B-53

ID	BENCHMARK	ACCEPTANCE CRITERION
B-51	**Vocabulary audit.** Enumerate every discharged object of Papers 0–11 against Appendix A; serialize the canonical document set; exercise every type.	100% closure in both directions; the canonical set validates at CIS-L1; every type instantiated.
B-52	**Conformance gauntlet.** The full suite of §10 executed against the reference build: oracle verdicts, trace audit, monitor battery with seeded defects, replay proof.	All four families pass on one build, one document set, one suite version — a single certificate, not a collage.
B-53	**Independent re-implementation check.** A second implementation, built from the CIS alone (specification-driven, not code-derived), is certified against the same suite.	Both implementations pass; every behavioral divergence between them is traceable to a specification ambiguity, which is then amended through the path — divergences are CIS defects, addressed to the documents.

PROOF OBLIGATION PO-14.1 — BUILD AND CERTIFICATION OF THE REFERENCE RUNTIME

Implement the runtime of Part III against the CIS of Part II; pass B-51 and B-52; then pass B-53 with an independent second implementation. Until PO-14.1 is discharged, the runtime is specified-but-unexecuted: H92–H98 stand as conjectures about a machine that does not yet run, and this document's cover notice — the reference runtime is specified, not yet executed — remains literally true.

Joins the open ledger: PO-9.1–9.3, PO-10.1, PO-12.1, PO-13.1. Sequencing: PO-14.1 feeds PO-13.1 (the ladder needs its harness and its ARM-5), and B-53's divergences feed the CIS amendment queue — the obligations pull each other forward, as the program's obligations always have.

13 Reconciliation with Papers 0–11 and PRIME

Stricter constraint wins; adoptions recorded. This paper adds the first *executable* machinery to the program, and the register's central entry is the discipline that keeps execution subordinate to specification — recorded formally as decision D-14.2: *where code and document disagree, the document governs and the code is defective*; the only repair path for a genuinely wrong document is the amendment path, never a patch FORMALIZED-HEREIN.

Table 13-1 — Reconciliation register — Paper 12 against Papers 0–11 and PRIME

PREDECESSOR	OBLIGATION OR TENSION	RESOLUTION IN THIS PAPER
Paper 0 — Constitutional Core	Invariants must be enforced, not described	Invariant monitors: Σ compiled to executable assertions on every transition (§8)
Paper 1 — Authority Algebra	Prohibited transitions unreachable	Register store rejects by construction; precondition checks are total functions (§8)
Paper 2 — Composition	Resolution total, logged, explainable	Precedence resolver emits certified cores; the resolution log is a trace table (§6, §8)
Paper 3 — Negative Space	Absence probes must fire in real time	Absence detectors run in stage 5 of the loop; AbsenceClass is a CIS type with executable probes
Paper 4 — Bounded Metacognition	Recursion bounded; transfer is mandatory	Audit loop's bound is a hard transition; transfer is a first-class trace event, not an exception
Paper 5 — Epistemic Governance	Provenance, decay, standing — for everything	Field-level hygiene in the CIS syntax; decay daemon; certificates as decaying standing
Paper 6 — Objective Transition	The register is asymmetric	ObjectiveSpace writes require human authorization events — enforced at the register store
Paper 7 — Survivability	Descent deterministic; recovery validated	Lattice controller with the integrity validator as its only re-ascent gate (§8)
Paper 8 — Feasibility	Admissibility is a fixed point, recomputed under shock	The gate evaluates A_{adm} against the current Endowment document — endowment amendment triggers recomputation (§6)
Paper 9 — Generative Lifecycle	Everything climbs; nothing shortcuts	CIS lineage slot; the runtime's own revisions enter at G (§11)
Paper 10 — Domain Modules	compat before execution	Lifecycle executor runs the COMPILE pipeline; ModuleDossier is the registry's serialized form
Paper 13 — Claim Architecture	The claim is a state machine (D-12.1)	The runtime is that state machine executed — Family A's mechanism, delivered as §6–§8
Paper 11 — Adversarial Instrument	The interposition API was recorded as owed; observables must be free	§7 pays the debt; every metric's observable is a contract-emitted event (§7)

PRIME architecture	One mechanism; the constitution applies to itself	Self-hosting (§11); H96's totality — no second tier of authority anywhere in the inventory

14 Output Contract

Table 14-1 — Output register — Paper 12

ARTIFACT	KIND	LOCATION	STATUS
CIS document model	Language specification	§3, Def. 3.1	FH
Object vocabulary	Twelve core types	§4, Appendix A	FH
Conformance levels	L0/L1/L2 with nesting	§5, Eq 5-1	FH
Main loop	The spine executed; gate predicate	§6, Eq 6-1/6-2	FH
Interposition contract	API + I-GATE	§7	FH
Component inventory	Eight components + monitors	§8	FH
Determinism substrate	Replay theorem + hash chain	§9, Eq 9-1	FH
Conformance suite	Four test families	§10	FH
H92–H98	Hypotheses	Table 12-1; B-51–B-53	CONJ
PO-14.1	Build obligation	§12	UNK

Downstream obligations: any implementation claiming the architecture must certify at CIS-L2 against the suite of §10 and publish the certificate with its document-set hash; any CIS amendment must re-run B-51's closure audit; any behavioral divergence between certified implementations is a CIS defect and must be amended as one; and any operational tooling introduced during the build of PO-14.1 must be registered as a governed artifact before first use — H96 is watching.

Related work and positioning are treated in Appendix RW (p. 32).

15 References and Program Colophon

Primary source. The SVT Architect Analysis file (Dr. Christabel Odeta — SAVANT / CLAI-OS / AEGIS-GLOBAL): the notation instruction ("perhaps a Constitutional Intelligence Specification (CIS)"); the runtime picture ("The model proposes. The kernel determines whether the proposal is admissible."); the publication doctrine naming the reference implementation as a separately published artifact; the decisive-move list requiring reproducible reference implementations and measurable failure modes; and the compression spine that this paper executes as a main loop.

Program genealogy. C1 (2026-09-30) Constitutional Core → C2 (2026-10-01) Authority Algebra → C3 (2026-10-02) Composition → C4 (2026-10-03) Negative Space → C5 (2026-10-06) Bounded Metacognition → C6 (2026-10-06) Epistemic Governance → C7 (2026-10-06) Objective Transition → C8 (2026-10-06) Survivability → C9 (2026-10-06) Resource-Constrained Intelligence → C10 (2026-10-06) Constitutional Generative Architecture → C11 (2026-10-06) Domain Modules → C12 (2026-10-06) The Claim Architecture → C13 (2026-10-06) The Adversarial Instrument → **C14 (2026-10-10) The Executable Constitution (this document)**.

Program ledger, consolidated. Hypotheses H1–H98; benchmarks B-01–B-53; outstanding proof obligations PO-9.1–9.3 (feasibility certificates), PO-10.1 (constitution sufficiency), PO-12.1 (prior-art and eligibility), PO-13.1 (ladder execution), PO-14.1 (reference build). The program now holds its theory, its runtime architecture, its modules, its claims, its instrument, and its machine-readable law — and it has kept the law's oldest property throughout: nothing here rules by being written down, everything rules by being discharged, and the machine that executes the constitution is governed by it like everything else.

Paper 12 of the SAVANT Constitutional Intelligence Series · the code is the shadow; the documents are the law · C0 informational · no running system delivered · 2026-10-10

A The CIS Core Schema Index

The complete serialization index: every core type with its fields, its reference floors (the minimum lifecycle state of referenced objects), and its monitor bindings (which compiled invariants watch it). This index is B-51's working surface; the audit passes when every discharged object of Papers 0–11 appears here exactly once and every type is exercised by the canonical document set.

Table A-1 — CIS core schema index — fields, reference floors, monitor bindings

TYPE	REQUIRED FIELDS	REFERENCE FLOORS	MONITOR BINDINGS
`Kernel`	invariant-refs, prohibition rules, objective-frame, version	self-contained (root type)	All of Σ — every invariant compiles against the Kernel's frame
`AuthorityRegister`	holder, level, scope, evidence-floor, expiry, obligations	Kernel ≥ A; EvidenceManifest ≥ A	Seven-operator legality; scope containment; expiry liveness
`PrecedenceOrder`	class lattice, tier map, resolution rules, refusal format	Kernel ≥ A	Totality (resolution or certified refusal); order consistency
`AbsenceClass`	probe spec, classification rule, contraction rule	Kernel ≥ A; AuthorityRegister schema	Probe liveness; contraction causality (detection ⇒ contraction event)
`AuditPolicy`	depth bound, stopping criteria, contradiction rules, transfer target	Kernel ≥ A	Bound enforcement; transfer-on-bound totality
`EvidenceManifest`	tier lattice, half-lives, provenance schema, floor rules	Kernel ≥ A	Floor gating; decay schedule firing; provenance completeness
`ObjectiveSpace`	objectives, legitimacy conditions, guard bindings, interim modes	Kernel ≥ A; human-principal registry	Register asymmetry (no autonomous writes); guard coverage
`TierLattice`	tiers, triggers, envelope tables, L3 artifact ref, validator spec	Kernel ≥ A	Invariant tier-independence; descent timing; validator gating
`Endowment`	constraint objects, default floors, PoI routing, custody log ref	Kernel ≥ A; EvidenceManifest ≥ A	Fixed-point recomputation on amendment; floor inviolability
`LifecycleObject`	state, Δ_{safe} conjuncts, dependency edges, notification log ref	Any (wraps all types)	auth = auth(state); conjunct discharge before transition; notifier completeness

`ModuleDossier`	nine documents per C11, registry entry, compat certificate ref	Kernel ≥ A; all nine slots ≥ V	I-MC: no runtime without compilation; slot completeness
`Scenario`	W, injection schedule, proposal stream, Ω-schema, G-reference	Corpus registry ≥ V	Observable completeness (S.4 frozen); ground-truth adjudication status
`ConformanceCertificate`	implementation hash, document-set hash, suite version, verdict records	Suite registry ≥ A	Expiry on amendment; single-certificate integrity (B-52)

Two index laws complete the appendix. **Exercise coverage:** the canonical document set — the program's own published corpus: kernel, five module dossiers, benchmark corpus sample, suite definitions — must instantiate every row; a type without an instance is either dead vocabulary (amend it out) or a missing artifact (build it), and B-51 decides which. **Floor totality:** every reference in the vocabulary carries a floor, so that the lifecycle's authority gradient is fully encoded in the type system: no parse can construct an authority that the path did not issue. The schema index is where Paper 9's law becomes a type error FORMALIZED-HEREIN.

B Decision Record D-14.1 — Policy Lives in Documents, Not Code

Decision. The reference runtime contains no policy: every rule it executes — every invariant, floor, threshold, precedence tier, decay rate, envelope, guard, and conjunct — is read from CIS documents. The code contributes execution machinery only: parsing, sequencing, evaluation, storage, emission.

Grounds. Four converging constraints. First, the amendment path: a rule in code is invisible to Paper 9's lifecycle — it climbs no gauntlet, notifies no dependents, decays on no schedule. Second, hygiene: Paper 5's provenance and decay are defined over documents; in-code rules inherit neither. Third, measurability: Paper 11's instrument measures document-sets across arms (P-BENCH); a rule in code is an arm difference that is not a document difference, and the measurement's semantics break. Fourth, the shadow doctrine (Def. 1.1): the reference implementation exists to make the specification executable; a second, hidden specification inside the code would make the executable shadow itself an object needing a shadow — regress without floor.

The escape valve, and its audit. Some choices are genuinely open in the specification — evaluation order among independent conjuncts, storage layout, probe scheduling within declared windows. Def. 1.1 requires these to be documented as choices; this decision makes the documentation concrete: every such choice is recorded in a `RuntimeProfile` document — a CIS object, versioned and amendable like any other. The valve exists; the valve is a document; and the document is on the path. There is no fourth place for policy to hide.

Consequences. (i) The runtime's correctness reduces to two separable questions — are the documents right (the instrument's question) and does the code execute the documents (the conformance suite's question) — and the reduction is H93's content. (ii) D-14.2 follows: where code and document disagree, the document governs and the code is defective; a wrong document is repaired only by amendment. (iii) The decision's cost is recorded honestly: document-driven execution forecloses implementation shortcuts that products routinely take (compiled-in fast paths, hardcoded thresholds), and the foreclosure is accepted as the price of auditability — the reference runtime is slow where slowness is the form fidelity takes.

Status. Adopted, 2026-10-10; binding on PO-14.1 and all derivative implementations claiming conformance; revisit only if a rule is discovered that cannot be expressed in the CIS vocabulary — which would falsify H92 and trigger a vocabulary amendment, not a code exception.

Appendix RW — Related Work and Positioning

This appendix is part of the elevation pass of 2026-10. It positions the paper against adjacent research programmes; the main text's structure, numbering, and reference list are unchanged. In-text citations follow the paper's author-year style; new references are listed at the end of this appendix.

Executable specifications and reference semantics. The reference discipline of §2 belongs to the tradition of executable formal semantics: a specification that runs, so that the gap between document and behavior is testable rather than rhetorical. Lamport's TLA+ work established the discipline's posture in concurrent systems: the specification is the artifact, model checking explores it, and implementations are judged against it (Lamport 2002). The CIS document model borrows that posture and adds a governance twist the systems tradition does not emphasize: where code and document disagree, the document governs and the code is defective, with the document itself repairable only by registered amendment. This inverts the normal authority of living code over stale documentation, and the inversion is the paper's thesis.

Policy languages and law as code. The nearest engineering relatives are the policy-as-code systems: XACML's standardized policy model, Open Policy Agent's declarative engine, and AWS's Cedar with its verification-guided design (OASIS 2013; Open Policy Agent; The Cedar Team 2024). These systems made rules executable, versioned, and analyzable. Catala (Merigoux et al.) went further in the other direction: a programming language derived from the structure of statutory law, in which the legislative text and the executable are one literate artifact (Merigoux et al. 2021). The CIS sits between these poles. Like the policy engines, it executes documents at runtime with an interposition contract (§7) guaranteeing no action bypasses the gate. Like Catala, it treats the document as the object of legal-grade scrutiny. Unlike both, its documents are constitutional: they carry provenance, expiry, standing, and amendment lineage (Papers 1–11), and conformance is behavioral (CIS-L2 requires execution against the suite, not inspection alone).

Constitutional AI and the meaning of "constitution." The phrase "constitutional AI" is now associated with Bai et al.'s training method, in which a written constitution guides self-critique during training (Bai et al. 2022); the RLHF substrate is Ouyang et al. (Ouyang et al. 2022). The terminological overlap obliges a clean separation. In Constitutional AI, the constitution is training signal: once training completes, the text has no operational role, and the trained model's behavior is the only artifact. In this series, the constitution is a runtime object: parsed, executed, amended, and audited. The two are compatible, and a deployment might use a constitutionally trained model inside a constitutionally governed runtime. But they are not the same claim, and this paper's conformance levels make the difference checkable: a CIS-L2 certificate is evidence about execution traces, which is a kind of evidence no training-time method currently produces.

Conformance and standards. The three-level conformance structure (§5) is designed to interface with external governance instruments; the crosswalk to NIST AI RMF, the EU AI Act, ISO/IEC 42001, and ISO 14971 is stated in §5's crosswalk paragraph. The design criterion throughout (L0 checkable on a laptop, L2 requiring execution) reflects a judgement about adoption: conformance machinery that requires trust in the vendor is not conformance machinery.

What this paper does not do. It does not deliver the runtime; PO-14.1 is a build obligation, and the conformance suite exists as specification, not as executed code. It does not claim CIS conformance implies safety; the levels certify that behavior matches documents, and the documents' worth is the burden of Papers 0–11 and of the empirical programme, all undischarged. It does not resolve the performance cost of document-driven execution; the reference discipline accepts slowness as the price of fidelity, and whether optimized runtimes can preserve the fidelity criterion is an open question, recorded. It does not cover multi-runtime deployments or distribution across administrative domains. And the self-hosting property of §11 (the runtime governing its own amendment) is stated with its circularity obligations as conjecture, pending the conformance suite it depends on.

References cited in this appendix

Lamport, L. (2002). Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. Addison-Wesley.

OASIS (2013). eXtensible Access Control Markup Language (XACML) Version 3.0.

Open Policy Agent. CNCF graduated project.

The Cedar Team (2024). Cedar policy language. Amazon Web Services.

Merigoux, D., Chataing, N., & Protzenko, J. (2021). Catala: A programming language for the law. Proceedings of the ACM on Programming Languages, 5(ICFP).

Bai, Y., et al. (2022). Constitutional AI: Harmlessness from AI feedback. arXiv:2212.08073.

Ouyang, L., et al. (2022). Training language models to follow instructions with human feedback. NeurIPS 2022.

Appendix H-J — §5.1 Standards Crosswalk

5.1 Standards crosswalk — the adoption path. The conformance levels are designed to be read against the instruments deployers and regulators already operate under. The mapping is directional and partial; CIS conformance does not by itself establish conformity with any listed instrument, and nothing here is legal advice. CIS-L0 (document validity) aligns with the documentation hygiene that the NIST AI Risk Management Framework's Govern function presupposes and that ISO/IEC 42001:2023 codifies as management-system practice: an AI system whose governing documents parse, type-check, and carry append-only lineage has the artifact discipline both frameworks ask organizations to maintain. CIS-L1 (static coherence) supplies evidence toward the EU AI Act's risk-management-system requirement (Regulation (EU) 2024/1689, Article 9) for systems whose Annex III classification brings them into scope — clinical modules being the obvious case — because L1 demonstrates that lifecycle floors, amendment histories, and composition closures have been checked across the whole document set rather than asserted. CIS-L2 (runtime conformance) is the level that speaks to Article 15's accuracy, robustness, and cybersecurity requirements and to the NIST AI RMF's Measure and Manage functions: the conformance suite's executed traces, oracle verdicts, and decaying certificates (each certificate states the document-set hash it certifies, and amendments expire dependents) are Measure-grade evidence with a Manage-grade maintenance loop attached. For clinical deployments, ISO 14971:2019's risk-management file is the natural container: the L2 suite's invariant monitors map to risk-control verification, and the oracle suite to the verification of

residual-risk acceptability, with the usual caveat that the standard's process obligations exceed what any conformance suite can evidence. The adoption path the levels encode is graduated on purpose: L0 is achievable with a parser and an afternoon, L1 with a checker over a document set, L2 with the reference runtime — so that a deployer can begin at documentation hygiene and climb, rather than facing the whole apparatus as a condition of entry. NIST's Generative AI Profile (AI 600-1) is the relevant companion reading where generated artifacts (Paper 9) enter the document set; the lifecycle gates of that paper supply the provenance and review evidence the profile asks for.